config.order = :random in the spec_helper.rb file to mandate random ordering of specs and to shake out potentially subtle test dependencies. Or you can use order --random on the RSpec command line to ensure random ordering.

The biggest impediment to independence in the test suite itself is the use of global data. Rails fixtures are not the only possible cause of global data in a Rails test suite, but they are a common cause. Somewhat less common in a Rails context is using a tool or third-party library in a setup and not tearing it down.

If you believe you have an intermittent test failure in RSpec that is triggered by the order of testing, the rspec --bisect option will attempt to discover the minimal set of specs that trigger the failure. I cover this in Chapter 15, *Troubleshooting and Debugging*, on page 303.

Outside the test suite, if the application code is not well encapsulated it may be difficult or impossible to make the tests fully independent.

Fast

It's easy to overlook the importance of pure speed in the long-term maintenance of a test suite or a TDD practice. In the beginning it doesn't make much difference. When you have only a few methods under test, the difference between one second per test and one-tenth of a second per test is almost imperceptible. The difference between a one-minute suite and a six-second suite is easier to discern.

From there, the sky's the limit. I worked in one Rails shop where nobody really knew how long the tests ran in development because they farmed out the test suite to an external server setup that was more powerful than most production web servers I've seen. This is bad.

Slow test suites hurt you in a variety of ways.

There are startup costs. In the sample TDD session you walked through in Chapter 2, *Test-Driven Development Basics*, on page 11, and Chapter 3, *Test-Driven Rails*, on page 41, you went back and forth to run the tests a lot. In practice I went back and forth even more frequently. Over the course of writing that tutorial, I ran the tests dozens of times. Imagine what happens if it takes even 10 seconds to start a test run. Or a minute, which is not out of the question for a larger Rails app. I've worked on JRuby-based applications that took well over a minute to start.

TDD is about flow in the moment, and the ability to go back and forth between running tests and writing code without breaking focus is crucial to being able